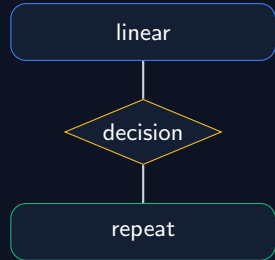


Introduction to Pseudocode

Computer Science Basics

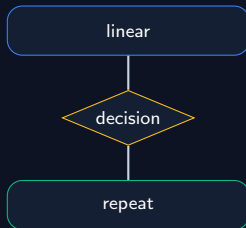


What is Pseudocode?

GETTING STARTED

Pseudocode is a language used to describe the steps of an algorithm.

- ▶ It borrows structures from real programming languages (Pascal, C/C++, BASIC, etc.), but it is meant to be read by people, not executed by computers.
- ▶ Implementation details are usually left out — such as variable declarations or language-specific syntax.
- ▶ Pseudocode can mix plain natural-language phrases with compact mathematical expressions that don't exist in real programming languages.



Core Concepts

GETTING STARTED

The concepts found in pseudocode algorithms are similar to those in real programming languages:

1

Instructions

2

Variables

3

Constants

4

Operations

5

Expressions

Many variants of pseudocode exist. This deck uses the variant found in Romanian Computer Science Baccalaureate exam problems — the same one taught in high-school algorithm courses.

Variables & Constants

BUILDING BLOCKS

Variables are never declared.

Their names follow the usual identifier rules: letters, digits, and the underscore character — and a name can never start with a digit.

Constants are literal values:

- ▶ integers and reals
- ▶ characters delimited with a single quote (')
- ▶ strings delimited with single (') or double (") quotes

Mathematical constants, such as π , may also appear directly in expressions.

```
// examples
```

```
age ← 15           // integer
```

```
gpa ← 3.85         // real
```

```
name ← 'Alex'      // string
```

```
grade ← 'A'        // character
```

Data Operations

BUILDING BLOCKS

Arithmetic Operations

Addition: $a + b$ — integer or real, depending on context

Subtraction: $a - b$

Multiplication: $a * b$

Division: a / b — always real; quotient floor: $[a/b]$

Remainder: $a \% b$ — integer only, integer result

Some variants use DIV and MOD instead. Exponentiation and square root may also appear.

Relational & Logical Operations

Relational:

$=, \neq, <, \leq, >, \geq$

Operands are usually numbers; the result is a boolean value (**true** or **false**).

Logical Operations:

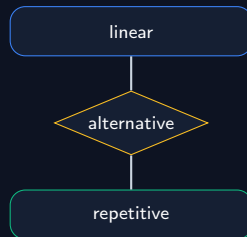
NOT, **AND**, **OR** — combine or invert boolean values.

Instructions: The Basics

BUILDING BLOCKS

Instructions are the parts of an algorithm that actually have an effect — they change variable values, read or display data, repeat actions, and so on.

- ▶ Usually each instruction is written on its own line (or several, for complex ones).
- ▶ To save space, two or more simple instructions can share a line, separated by ;.
- ▶ Every algorithm can be built from three kinds of structure:
 - ▶ **linear**
 - ▶ **alternative** (decision)
 - ▶ **repetitive**



Reading, Writing & Assignment

The three instructions every pseudocode algorithm starts with.

Reading Data

DEMO 1

```
read <variable list>
```

<variable list> is a sequence of variables separated by commas.

- ▶ Successive values are taken from the keyboard, in order, and stored in the listed variables.
- ▶ A `read` instruction is usually paired with notes about the expected data — its type, valid range, and so on.

```
read a, b, c
```

Writing Data

DEMO 2

```
write <expression list>
```

<expression list> is a sequence of expressions separated by commas.

- ▶ The expressions are evaluated in order, and their results are printed to the screen.

```
write 'a + b = ', a + b  
  
// if a = 3 and b = 4:  
// a + b = 7
```

The Assignment Instruction

DEMO 3

$\text{<variable>} \leftarrow \text{<expression>}$

The expression is evaluated first, and the resulting value is then stored in the variable.

- ▶ This is the workhorse instruction behind every calculation in a pseudocode algorithm.

```
a ← 0  
S ← a + b  
i ← i + 1
```

The Alternative Structure

DECISION MAKING

Execution order:

1. Evaluate `<condition>`.
2. If it is true, execute `<instructions1>`, then move on to the next instruction.
3. If it is false, execute `<instructions2>` — when an else branch exists — then move on to the next instruction.

```
┌ if <condition> then  
|   <instructions1>  
| else  
|   <instructions2>  
└ ■
```

```
┌ if <condition> then  
|   <instructions1>  
└ ■
```

Repetitive Structures

Three ways to repeat a sequence of instructions.

FOR

known number
of steps

WHILE

unknown steps,
initial test

DO...WHILE

unknown steps,
final test

Classification of Repetitive Structures

TAXONOMY & CRITERIA

By Test Placement

1. Initial Test (Pre-condition)

- ▶ Tested *before* loop body
- ▶ Executions: **0 or more times**
- ▶ **WHILE**

2. Final Test (Post-condition)

- ▶ Tested *after* loop body
- ▶ Executions: **at least 1 time**
- ▶ **DO...WHILE**

By Number of Iterations

1. Known Number of Steps

- ▶ Determined beforehand (bounded)
- ▶ **FOR**

2. Unknown Number of Steps

- ▶ Condition-dependent (unbounded)
- ▶ **WHILE**
- ▶ **DO...WHILE**

The FOR Loop

REPETITIVE STRUCTURES — KNOWN STEPS

```
┌ for <var> ← <start>, <end>, <step>
  do
  |   <instructions>
└ ──
```

1. start, end and step evaluate to integers;
2. step is 1 or -1 , defaulting to 1 when omitted.
3. If $\text{step} = 1$ and $\text{start} > \text{end}$, the instructions never run.

Demo — sum & product of first n numbers

```
S ← 0; P ← 1
┌ for i ← 1, n do
  |   S ← S + i
  |   P ← P * i
└ ──
write S, ' ', P
```

The WHILE Loop

REPETITIVE STRUCTURES — INITIAL TEST

```
┌ while <condition> do  
│   <instructions>  
└ ─
```

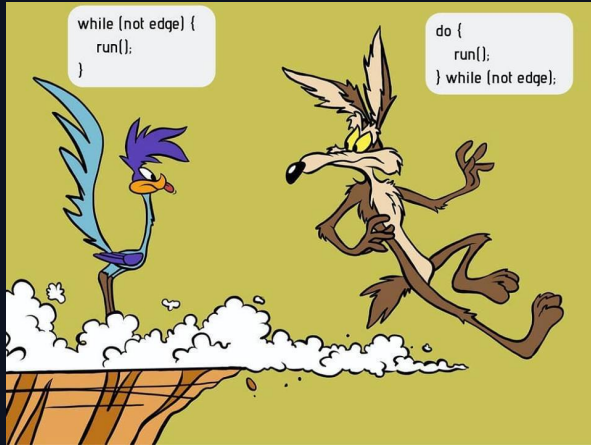
1. Evaluate <condition>.
2. If true, run instructions and go back to step 1.
3. If false, move on to the next instruction.

Demo — sum of the digits of a number

```
read n  
S ← 0  
┌ while n ≠ 0 do  
│   S ← S + n % 10  
│   n ← [n / 10]  
└ ─  
write S
```

DO...WHILE LOOP

WHILE VS DO-WHILE



The DO...WHILE Loop

REPETITIVE STRUCTURES — FINAL TEST

```
┌ do
|   <instructions>
└ while <condition>
```

1. Run the instructions.
2. Evaluate <condition>.
3. If true, go back to step 1. If false, move on.

Important: Even if the condition is **false** from the **start**, the instructions RUN **at least once**.

Demo — number of digits of a number

```
read n
cnt ← 0
┌ do
|   cnt ← cnt + 1
|   n ← [n / 10]
└ while n ≠ 0
write cnt
```

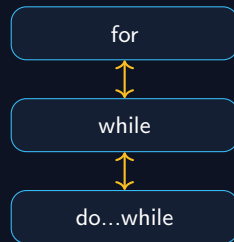
Equivalence of Repetitive Structures

REPETITIVE STRUCTURES

Many exercises give you an algorithm and ask for an equivalent one that uses a different repetitive structure.

Two algorithms are equivalent if, for every set of valid input data, they produce exactly the same results.

The next slides walk through the standard transformation rules between FOR, WHILE, and DO...WHILE.



FOR $\rightarrow$ WHILE

STRUCTURE EQUIVALENCE

Before

```
┌ for i  $\leftarrow$  a, b do  
|   <instructions>  
└ ─
```

Equivalent

```
i  $\leftarrow$  a  
┌ while i  $\leq$  b do  
|   <instructions>  
|   i  $\leftarrow$  i + 1  
└ ─
```

FOR $\rightarrow$ DO...WHILE

STRUCTURE EQUIVALENCE

Before

```
┌ for i  $\leftarrow$  a, b do  
|   <instructions>  
└ ─
```

Equivalent

```
i  $\leftarrow$  a  
┌ if i  $\leq$  b then  
| ┌ do  
| |   <instructions>  
| |   i  $\leftarrow$  i + 1  
| └ while i  $\leq$  b  
└ ─
```

WHILE → DO...WHILE

STRUCTURE EQUIVALENCE

Before

```
┌ while <condition> do  
│   <instructions>  
└ ■
```

Equivalent

```
┌ if <condition> then  
│ ┌ do  
│ │   <instructions>  
│ └ while <condition>  
└ ■
```

Questions?

Next up: Lab time !! Introduction to Python and VS Code.

Thank you for your attention!